AI Audit Report — HW02 Domain Testing

1. Thông tin chung

- **Bài tập:** HW02 — Domain Testing on EShop.
- **Ngày ghi nhận:** 14/10/2026 và 16/10/2026.
- **Thời gian:** Một số tương tác có timestamp cụ thể; các mốc chưa xác minh được ghi rõ trong từng mục, không tự suy đoán.
- **Công cụ AI:** ChatGPT của OpenAI và GitHub Copilot trong VS Code.
- **Phạm vi sử dụng AI:** Phân tích yêu cầu; tổ chức repository; thiết kế Domain Testing và BVA cho các feature; hỗ trợ kiểm thử tự động/thủ công; sắp xếp evidence; lập bug report và AI Gap Analysis; tạo, kiểm tra Agent Skill; hỗ trợ hoàn thiện tài liệu.
- **Nguyên tắc sử dụng:** Đầu ra AI là tài liệu hỗ trợ, không mặc nhiên được xem là đúng hoặc là kết quả thực thi. Người học đối chiếu với SRS, xác minh kết quả, rà soát bằng chứng và chỉnh sửa những nội dung chưa chính xác.
- **Lưu ý về nhật ký:** Các prompt/đầu ra dài được tóm tắt hoặc trích nguyên văn tùy khả năng truy cập lịch sử. Mục nào thiếu dữ liệu sẽ nêu rõ giới hạn. Interaction ID được giữ để truy vết; do nhật ký được nối từ nhiều session, ID không nhất thiết phản ánh thứ tự thời gian tuyệt đối.

---

2. Nhật ký sử dụng ChatGPT

Lần 1. Phân tích yêu cầu bài tập và lựa chọn feature

Thời gian: Ngày 14/10/2026; giờ cụ thể cần đối chiếu lịch sử ChatGPT.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> Giải thích chi tiết bài tập sau, thì tui chọn các FR 1,9,17 tương ứng cho 3 pool đầu pool 4 thì sử dụng mobile test cô quy định đó là các chức năng bao gồm 1->19 thuộc 3 pool còn lại nhưng không được chọn trùng thì tui chọn FR 15 cho pool 4.

Tóm tắt đầu ra AI:

- Phân tích yêu cầu HW02 và hướng dẫn quy trình áp dụng Domain Testing, BVA cho các feature được chọn.
- Xác định FR-01 là đăng ký tài khoản, FR-09 là mã giảm giá và FR-17 là quản lý mã giảm giá.
- Lưu ý cần xác minh phạm vi FR-15 khi kiểm thử Mobile, vì chức năng quản lý sản phẩm trên Web Admin không mặc nhiên có mặt trên ứng dụng Mobile.
- Đề xuất cấu trúc báo cáo, cách ghi nhận kết quả thực thi, bằng chứng, bug report, AI Gap Analysis, AI Audit Report và Git commit log.

Rà soát của người dùng: Chọn FR-01, FR-09 và FR-17 cho ba pool đầu. Phạm vi FR-15 trên Mobile cần được xác minh với hướng dẫn của giảng viên và chức năng thực tế của ứng dụng.

Lần 2. Hỗ trợ tạo cấu trúc repository HW02

Thời gian: Khoảng 21:43 ngày 14/10/2026 (UTC+7).

Công cụ AI: ChatGPT.

Prompt của người dùng:

> oke tui đã git clone ebi shop về tiếp theo mình sẽ tạo repo với cấu trúc sau HW02/ ... có cách nào tạo nhanh ko

Tóm tắt đầu ra AI:

- Đề xuất tạo repository bài tập HW02 riêng, tách biệt với repository mã nguồn EShop SUT.
- Cung cấp script PowerShell tạo các thư mục `features`, `bug-reports`, `evidence`, `agent-skills` và các file Markdown cần thiết.
- Hướng dẫn khởi tạo Git repository bằng `git init`.
- Giải thích cách kiểm tra cấu trúc thư mục bằng lệnh `tree /F`.

Rà soát của người dùng: Sử dụng script để tạo cấu trúc ban đầu; kiểm tra lại đường dẫn và nội dung file sau khi tạo.

Lần 3. Khắc phục việc không thấy repository trên GitHub

Thời gian: Ngày 14/10/2026; giờ cụ thể cần đối chiếu lịch sử ChatGPT.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> Sao tui ko thấy trên git ta

Tóm tắt đầu ra AI:

- Giải thích rằng `git init` chỉ tạo Git repository trên máy tính, không tự động tải dữ liệu lên GitHub.
- Hướng dẫn tạo repository mới trên website GitHub.
- Hướng dẫn kết nối repository local với remote bằng `git remote add origin`.
- Hướng dẫn kiểm tra trạng thái Git, commit các file và push nhánh `main` lên GitHub.

Rà soát của người dùng: Phân biệt repository local với repository từ xa và thực hiện push sau khi cấu hình remote phù hợp.

Lần 4. Xác định các lớp tương đương của FR-01

Thời gian: Ngày 14/10/2026; giờ cụ thể cần đối chiếu lịch sử ChatGPT.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> Dựa trên đặc tả FR-01 được cung cấp, chỉ xác định input variables, output variables và điều kiện nghiệp vụ. Chưa tạo test case. Với mỗi kết luận, chỉ ra câu trong đặc tả làm căn cứ. Nêu rõ những thông tin chưa được quy định tương tự dựa trên input/output và đặc tả đã phân tích, thực hiện Step 2 Domain Testing. Lập bảng điều kiện, mã EC, mô tả miền Valid và Invalid. Không tự thêm ràng buộc không có trong đặc tả. Tách lớp khi có khả năng được xử lý khác nhau. Trình bày giống slide đã gửi

Tóm tắt đầu ra AI:

- Hướng dẫn thực hiện Bước 1 của Domain Testing: xác định biến đầu vào và đầu ra.
- Ban đầu, AI cung cấp bản nháp tiếng Anh gồm các biến họ tên, email, mật khẩu và xác nhận mật khẩu.
- Đề xuất ghi nhận các kết quả đầu ra như kết quả đăng ký, thông báo kiểm tra dữ liệu và chuyển trang sau khi đăng ký thành công.
- Giải thích sự khác biệt giữa biến đầu vào và điều kiện đầu vào.
- Hướng dẫn Bước 2 — Identify Equivalence Classes theo tài liệu Domain Testing.
- Phân chia các điều kiện của FR-01 thành các lớp hợp lệ (Valid) và không hợp lệ (Invalid).
- Đánh mã EC01 đến EC19, bao gồm họ tên, email, độ dài mật khẩu, chữ hoa, chữ thường, chữ số, ký tự đặc biệt và xác nhận mật khẩu.
- Lập bảng các lớp tương đương cho từng điều kiện và bảng tổng hợp các phân hoạch.
- Giải thích việc tách các lớp Invalid để thuận lợi cho kiểm thử từng điều kiện riêng biệt.

Rà soát của người dùng: Sử dụng bảng EC làm cơ sở cho bước lựa chọn test case. Cần đối chiếu các điều kiện và tập ký tự được chấp nhận với đặc tả chính thức của EShop.

Lần 5. Lựa chọn test case đại diện cho các lớp tương đương

Thời gian: Ngày 14/10/2026; giờ cụ thể cần đối chiếu lịch sử ChatGPT.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> Dựa trên bảng EC đã review, thực hiện Step 3. Chọn bộ test case đại diện, cho biết mỗi TC bao phủ EC nào. Các TC Invalid chỉ nên vi phạm một lớp Invalid tại một thời điểm. Giải thích vì sao tập test được chọn có thể bao phủ các lớp đã xác định.

Tóm tắt đầu ra AI:

- Hướng dẫn Bước 3 — Selecting Test Cases.
- Đề xuất bộ test case ban đầu gồm 12 test case cho FR-01.
- TC01 kiểm tra đăng ký với dữ liệu hợp lệ và bao phủ các lớp Valid.
- TC02–TC12 kiểm tra lần lượt các lớp Invalid như họ tên rỗng, email rỗng, email sai định dạng, email trùng lặp, mật khẩu thiếu điều kiện và xác nhận mật khẩu không hợp lệ.
- Trình bày dữ liệu đầu vào, mục đích kiểm thử, lớp EC được bao phủ và kết quả mong đợi.
- Đề xuất cách kiểm tra độ bao phủ của các lớp EC.

Rà soát của người dùng: Sử dụng bộ 12 test case làm bản thiết kế ban đầu. Các giá trị email và điều kiện môi trường cần được chuẩn bị trước khi thực thi. Kết quả thực tế chưa được xác nhận chỉ từ việc thiết kế test case.

Lần 6. Áp dụng Boundary Value Analysis cho FR-01

Thời gian: Ngày 14/10/2026; giờ cụ thể cần đối chiếu lịch sử ChatGPT.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> Thực hiện Step 4 Boundary Value Analysis trên các miền giá trị có biên được đặc tả. Xác định biên dưới, biên trên nếu có và chọn các giá trị ngay dưới, tại và ngay trên biên. Phân biệt test case thuộc BVA với test case chỉ dùng để kiểm tra lớp tương đương.

Tóm tắt đầu ra AI:

- Hướng dẫn tạo file `features/FR-01-registration/bva.md`.
- Xác định giới hạn độ dài mật khẩu tối thiểu là 8 ký tự.
- Thiết kế ba test case kiểm tra các giá trị 7, 8 và 9 ký tự.
- Giải thích cách giữ các điều kiện khác hợp lệ để tập trung kiểm tra riêng biên độ dài.
- Bổ sung bảng ghi nhận kết quả mong đợi, kết quả thực tế, trạng thái và bằng chứng.
- Hướng dẫn commit phần BVA bằng Git.

Rà soát của người dùng: Bộ test case BVA được xem là bộ thiết kế ban đầu. Cần thực thi trên EShop để xác nhận Pass/Fail và thu thập bằng chứng.

Lần 7. Tổng hợp AI Audit Report

Thời gian: 22:57 ngày 14/10/2026 (UTC+7), theo mốc thời gian của phiên làm việc hiện tại.

Công cụ AI: ChatGPT.

Prompt của người dùng:

> bạn cho tui nội dung ai-audit + time stampe khi mà làm từ nãy đến giờ để tui dán vô

Tóm tắt đầu ra AI:

- Tổng hợp nhật ký sử dụng AI theo từng giai đoạn đã thực hiện.
- Ghi nhận prompt, tóm tắt đầu ra AI, kết quả review và các mốc thời gian xác định được.
- Đánh dấu những thời điểm chưa có đủ thông tin để xác minh chính xác, tránh tự tạo timestamp không có căn cứ.

Lần 8. Áp dụng Domain Testing và Boundary Value Analysis cho FR-09

Công cụ AI: ChatGPT (OpenAI)

Thời gian: 14/10/2026

Thời gian: Sau 22:57, giờ Việt Nam (UTC+7). Cần đối chiếu lịch sử ChatGPT để xác định chính xác giờ và phút.

Mục tiêu sử dụng AI: Hỗ trợ phân tích và thiết kế bộ test case cho FR-09 – Discount Coupons thuộc Pool B (Shopping Cart and Checkout) theo đúng trình tự Domain Testing và Boundary Value Analysis được giảng dạy trong môn Kiểm thử phần mềm.

Ghi chú: Lần này sử dụng prompt tổ hợp đã dùng khi làm FR-01.

1. Prompt đã sử dụng

Prompt 1 – Xác định Input/Output và điều kiện nghiệp vụ

> Dựa trên đặc tả FR-09 được cung cấp, chỉ xác định input variables, output variables và điều kiện nghiệp vụ. Chưa tạo test case. Với mỗi kết luận, chỉ ra câu trong đặc tả làm căn cứ. Nêu rõ những thông tin chưa được quy định.

Prompt 2 – Xác định các lớp tương đương

> Dựa trên input/output và đặc tả đã phân tích, thực hiện Step 2 Domain Testing. Lập bảng điều kiện, mã EC, mô tả miền Valid và Invalid. Không tự thêm ràng buộc không có trong đặc tả. Tách lớp khi có khả năng được xử lý khác nhau.

Prompt 3 – Lựa chọn test case đại diện

> Dựa trên bảng EC đã review, thực hiện Step 3. Chọn bộ test case đại diện, cho biết mỗi TC bao phủ EC nào. Các TC Invalid chỉ nên vi phạm một lớp Invalid tại một thời điểm. Giải thích vì sao tập test được chọn có thể bao phủ các lớp đã xác định.

Prompt 4 – Phân tích giá trị biên

> Thực hiện Step 4 Boundary Value Analysis trên các miền giá trị có biên được đặc tả. Xác định biên dưới, biên trên nếu có và chọn các giá trị ngay dưới, tại và ngay trên biên. Phân biệt test case thuộc BVA với test case chỉ dùng để kiểm tra lớp tương đương. Làm tương tự với FR-09 thuộc Pool 2, đồng thời ghi chú rõ từng phần cần dán vào file nào.

2. Đầu ra của AI

AI đã hỗ trợ xây dựng nội dung cho hai file:

File 1: `features/FR-09-coupons/domain-testing.md`

- **Bước 1 – Xác định biến đầu vào và đầu ra:** Xác định mã giảm giá, tổng tiền đơn hàng, trạng thái đăng nhập, trạng thái hoạt động, ngày hết hạn, ngưỡng đơn hàng, loại mã, giá trị giảm và giới hạn lượt sử dụng.
- Xác định năm điều kiện nghiệp vụ: mã tồn tại và đang hoạt động; mã còn hạn; tổng tiền đạt ngưỡng tối thiểu; người dùng có JWT hợp lệ; số lượt đã sử dụng chưa đạt giới hạn.
- Xác định kết quả áp dụng mã, số tiền giảm, số tiền cuối cùng và phản hồi của hệ thống.
- Ghi nhận các thông tin chưa được quy định rõ trong đặc tả, chẳng hạn quy tắc phân biệt chữ hoa/chữ thường của mã, làm tròn số tiền giảm, múi giờ và thời điểm tính lượt sử dụng.
- **Bước 2 – Xác định lớp tương đương:** Xây dựng các lớp EC01–EC12 cho năm điều kiện nghiệp vụ và EC13–EC14 cho hai nhánh tính toán `percent` và `fixed`.
- **Bước 3 – Lựa chọn test case:** Đề xuất 9 test case ban đầu, bao gồm áp dụng mã thành công, mã không tồn tại, mã không hoạt động, mã hết hạn, đơn hàng chưa đạt ngưỡng, JWT không hợp lệ và trường hợp đã sử dụng hết lượt.
- Kiểm tra khả năng bao phủ các lớp tương đương và yêu cầu mỗi test case Invalid chỉ vi phạm một điều kiện tại một thời điểm.

File 2: `features/FR-09-coupons/bva.md`

- **Bước 4 – Boundary Value Analysis:** Xác định ba nhóm biên có thể kiểm tra là ngưỡng đơn hàng tối thiểu, hạn sử dụng và giới hạn lượt dùng.
- Đề xuất 9 test case BVA, sử dụng các giá trị ngay dưới, tại và ngay trên từng biên.
- Bổ sung bảng kết quả mong đợi, kết quả thực tế, trạng thái và bằng chứng.
- Phân biệt các điều kiện đã được kiểm tra bằng lớp tương đương với các điều kiện có thể kiểm tra thông qua giá trị biên.
- Hướng dẫn lưu file và tạo Git commit riêng cho Domain Testing và BVA.

3. Kết quả rà soát của sinh viên

Đầu ra AI được sử dụng làm bản thiết kế ban đầu, chưa được xem là bộ test case đã xác nhận đúng hoàn toàn hoặc đã thực thi thành công.

Các nội dung cần kiểm tra lại gồm:

1. Đối chiếu từng điều kiện nghiệp vụ với đặc tả FR-09 trong SRS chính thức.
2. Xác minh dữ liệu của các mã `SAVE10`, `BIGBUY`, `VIP100` và `EXPIRED` trong môi trường kiểm thử.
3. Kiểm tra trạng thái hoạt động, ngày hết hạn, ngưỡng đơn hàng và giới hạn lượt sử dụng thực tế của từng mã trước khi chạy test.
4. Xác nhận cách hệ thống xử lý ngày hết hạn, đơn vị tiền tệ và số lượt đã sử dụng.
5. Chỉ ghi nhận Pass hoặc Fail sau khi thực thi và so sánh kết quả thực tế với kết quả mong đợi.
6. Ghi nhận bằng chứng và tạo GitHub Issue nếu xác minh được lỗi của hệ thống.

4. Đánh giá sơ bộ việc sử dụng AI

AI hỗ trợ tổ chức quy trình kiểm thử theo từng bước, giảm công sức lập bảng phân hoạch và lựa chọn các giá trị đại diện. Tuy nhiên, kết quả vẫn phụ thuộc vào tính đầy đủ của đặc tả và dữ liệu kiểm thử.

Sinh viên cần chủ động xác minh các giả định, rà soát độ bao phủ của EC, kiểm tra các điều kiện biên và thực thi trên ứng dụng thực tế. Những thông tin chưa được quy định trong SRS không được tự động xem là yêu cầu bắt buộc của hệ thống.

Trạng thái công việc: Đã thiết kế bản nháp Domain Testing và BVA cho FR-09; kết quả thực thi và bằng chứng cần được cập nhật sau khi kiểm thử ứng dụng EShop.

---

3. Nhật ký GitHub Copilot — nhóm tương tác COPILOT-01–COPILOT-10

Các mục dưới đây được nối thêm từ lịch sử hội thoại và các thao tác/file được xác nhận trong workspace. Ngày của phiên là 16/10/2026. Với các lượt không có timestamp kèm theo trong lịch sử khả dụng, giờ được ghi là **Không xác định được** thay vì ước lượng. Prompt dài được giữ nguyên trong phần chi tiết; đầu ra AI được tóm tắt trung thực.

Bảng tổng quan

Mã tương tác	Ngày/giờ (UTC+7)	Công cụ AI	Nhiệm vụ	Kết quả thực tế	Rà soát của người dùng	Tài liệu liên quan
COPILOT-01	16/10/2026; giờ không xác định được	GitHub Copilot trong VS Code; Skill `eshop-domain-bva-testing`	Thử áp dụng thực tế Skill để phân tích FR-01	Skill được gọi; tạo tài liệu kiểm chứng mới sau khi tìm thấy SRS gốc và đối chiếu test design/evidence	Người dùng yêu cầu không sửa các file test gốc, source, database hoặc execution/evidence; giới hạn được tuân thủ	`agent-skills/demo/FR-01-skill-validation.md`

Mã tương tác	Ngày/giờ (UTC+7)	Công cụ AI	Nhiệm vụ	Kết quả thực tế	Rà soát của người dùng	Tài liệu liên quan
COPILOT-02	16/10/2026; giờ không xác định được	GitHub Copilot trong VS Code	Hoàn thiện demo Skill FR-01 theo SRS	Cập nhật demo Domain/BVA và ghi chú kiểm chứng để phân biệt SRS, API specification, giả định và test execution	Đối chiếu yêu cầu với SRS gốc; không sửa test plan/execution/evidence gốc	`agent-skills/demo/FR-01-domain-bva-analysis.md`, `agent-skills/demo/FR-01-skill-validation.md`
COPILOT-03	16/10/2026; giờ không xác định được	GitHub Copilot trong VS Code	Rà soát và lập Bug Report FR-01/09/17/11 từ kết quả thực tế	Tạo index và 10 báo cáo; nhóm 27 ca FAIL theo nguyên nhân; không tạo bug từ ca BLOCKED; không đăng issue	Đối chiếu SRS, execution, evidence, traces và screenshots; severity để chưa đánh giá khi thiếu rubric	`bug-reports/README.md`, `bug-reports/BR-*.md`
COPILOT-04	16/10/2026 21:55:18 (UTC+7)	GitHub Copilot trong VS Code	Việt hóa Bug Reports sau phản hồi người dùng	Việt hóa README tổng hợp và cả 10 file báo cáo; giữ nguyên ID, dữ liệu kiểm thử và liên kết evidence	Kiểm tra liên kết Markdown; không thay đổi kết quả hoặc evidence	`bug-reports/README.md`, `bug-reports/BR-*.md`
COPILOT-05	16/10/2026 22:19:22 (UTC+7)	GitHub Copilot trong VS Code	Thực hiện AI Gap Analysis bốn feature	Tạo báo cáo mới với 12 phát hiện; ghi rõ giới hạn attribution cho FR-17/FR-11; kiểm tra liên kết và cấu trúc bảng	Yêu cầu phân biệt lỗi ứng dụng, thiếu sót test design, giả định không có căn cứ và ca BLOCKED	`ai-gap-analysis.md`
COPILOT-06	16/10/2026 22:30:48 (UTC+7)	GitHub Copilot trong VS Code	Cập nhật AI Audit nối tiếp cho session	Đọc audit hiện có trước khi sửa; nối thêm nhật ký các tương tác mới, không ghi đè phần lịch sử 14/10	Giữ prompt nguyên văn khi còn truy cập được; đánh dấu giờ không xác định; xác nhận chỉ audit được sửa trong lượt này	`ai-audit.md`

Chi tiết tương tác

COPILOT-01 — Áp dụng Skill thực tế cho FR-01

- **Thời gian:** 16/10/2026; giờ chính xác không xác định được từ lịch sử khả dụng.
- **Công cụ AI:** GitHub Copilot trong VS Code; Skill `eshop-domain-bva-testing` được gọi trong phiên.
- **Prompt của người dùng (nguyên văn):**

> /eshop-domain-bva-testing Hãy áp dụng Skill `/eshop-domain-bva-testing` vừa gọi để thực hiện một lượt phân tích thực tế cho FR-01 Registration.
>
> 1. Tìm đặc tả FR-01 gốc trong repo `eshop-sut`, ưu tiên kiểm tra `api_specification.md` và `README.md` để xác định nguồn chính xác.
> 2. Dùng Skill để thực hiện Domain Testing và BVA theo đúng quy trình trong `SKILL.md`.
> 3. Đối chiếu kết quả với các file Domain Testing và BVA FR-01 đã có trong repo HW02. Nêu những điểm trùng, thiếu hoặc cần chỉnh sửa, không tự ý sửa các file cũ.
> 4. Lưu kết quả kiểm chứng mới vào `HW02/agent-skills/demo/FR-01-skill-validation.md`.
> 5. Không sửa source code EShop, database, test plan, execution results hoặc evidence. Không commit hoặc push.
> 6. Cuối cùng, xác nhận Skill đã được gọi thực tế hay chưa, những file nào đã đọc và file demo nào đã tạo. Nếu không tìm thấy đặc tả gốc hoặc không gọi được Skill, hãy báo rõ, không giả vờ đã hoàn thành.

- **Tóm tắt đầu ra AI:** Skill được gọi thực tế; báo cáo kiểm chứng được tạo, ghi nguồn FR-01 và so sánh với tài liệu Domain Testing/BVA hiện có. Không sửa file gốc hay chạy test trong lượt áp dụng Skill.
- **Tệp và thao tác:** Tạo `agent-skills/demo/FR-01-skill-validation.md`; đọc SRS/API specification, test design và Skill liên quan theo yêu cầu.
- **Kết quả thực tế:** File kiểm chứng được lưu trong repo. Việc gọi Skill được xác nhận trong đầu ra phiên, không chỉ được đề xuất.
- **Rà soát của người dùng:** Người dùng yêu cầu rõ không chỉnh sửa test plan/execution/evidence cũ; phạm vi đầu ra chỉ là demo mới.

- **Giới hạn/chưa xác minh:** Không ghi timestamp đến phút vì không có căn cứ. Lịch sử không phải transcript đầy đủ của toàn bộ đầu ra.

COPILOT-02 — Hoàn thiện tài liệu demo Skill FR-01

- **Thời gian:** 16/10/2026; giờ chính xác không xác định được.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> Hãy hoàn thiện tài liệu demo Agent Skill sau lượt áp dụng thực tế vừa thực hiện.
>
> Nguồn đối chiếu:
>
> - `agent-skills/demo/FR-01-domain-bva-analysis.md`
> - `agent-skills/demo/FR-01-skill-validation.md`
> - `.github/skills/eshop-domain-bva-testing/SKILL.md`
> - Đặc tả SRS trong `eshop-sut/README.md`
> - `eshop-sut/api_specification.md`
>
> Yêu cầu:
>
> 1. Cập nhật `FR-01-domain-bva-analysis.md` để phản ánh việc đã tìm thấy và đọc SRS gốc trong `eshop-sut/README.md`. Xóa hoặc sửa nhận định cũ rằng không tìm thấy đặc tả gốc.
> 2. Giữ nguyên mục đích của file này là đầu ra thiết kế minh họa, không phải kết quả thực thi.
> 3. Đối chiếu điều kiện, phân lớp tương đương, test case và BVA với SRS. Chỉ sửa các nội dung demo chưa nhất quán với nguồn chính thức.
> 4. Không tự ý thay đổi các file Domain Testing, BVA, execution results và evidence gốc.
> 5. Giữ rõ khác biệt giữa SRS, API specification và giả định chưa được quy định. Không tự đặt thông báo lỗi hoặc hành vi API chưa có căn cứ.
> 6. Kiểm tra liên kết Markdown và đảm bảo hai file demo không mâu thuẫn nhau.
> 7. Báo lại các nội dung đã thay đổi và những điểm vẫn cần con người xác minh. Không chạy lại bộ test, không sửa source code EShop, không sửa database và không commit/push.

- **Tóm tắt đầu ra AI:** Sửa demo FR-01 để lấy SRS gốc làm nguồn nghiệp vụ ưu tiên, tách contract API khỏi yêu cầu SRS, gộp xác nhận rỗng/không khớp thành một lớp hành vi nhưng giữ các biến thể test, và tránh tự đặt message/response lỗi. Ghi rõ đây là thiết kế minh họa, không phải execution.
- **Tệp và thao tác:** Cập nhật `agent-skills/demo/FR-01-domain-bva-analysis.md` và `agent-skills/demo/FR-01-skill-validation.md`; kiểm tra các liên kết cục bộ của hai demo.
- **Kết quả thực tế:** Các liên kết cục bộ được báo cáo là hợp lệ. Không chạy test và không sửa tài liệu test gốc.
- **Rà soát của người dùng:** Người dùng giới hạn thay đổi vào hai file demo và yêu cầu nêu những điểm còn cần xác minh.
- **Giới hạn/chưa xác minh:** Không xác minh độc lập lịch sử chỉnh sửa trước phiên; chỉ ghi nhận thay đổi ở demo trong lượt này.

COPILOT-03 — Rà soát và tạo Bug Reports cho bốn feature

- **Thời gian:** 16/10/2026; giờ chính xác không xác định được.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # NHIỆM VỤ: RÀ SOÁT VÀ HOÀN THIỆN BUG REPORT CHO HW02
>
> Hãy hoàn thiện Bug Report cho bốn feature FR-01, FR-09, FR-17 và FR-11 dựa trên kết quả kiểm thử thực tế hiện có.
>
> ## 1. Đọc các nguồn liên quan
>
> Tìm và đọc:
>
> - Domain Testing và BVA của cả bốn feature.
> - `execution-results.md` của FR-01, FR-09, FR-17 và FR-11.
> - Các thư mục evidence, `evidence-index.md`, screenshot và log liên quan.
> - SRS trong repo EShop.
> - Các bug report đã tồn tại trong repo HW02, nếu có. Tìm file theo cấu trúc workspace thực tế, không giả định tất cả tài liệu đều nằm ở một vị trí cố định.
>
> ## 2. Xác định lỗi thực sự
>
> Đối với từng test case FAIL, hãy:
>
> 1. Đối chiếu expected result với actual result.
> 2. Kiểm tra evidence và log để xác định có đủ căn cứ kết luận lỗi ứng dụng hay không.

3. Phân biệt lỗi ứng dụng với lỗi test script, môi trường hoặc dữ liệu kiểm thử.
4. Đối chiếu với SRS trước khi kết luận ứng dụng vi phạm yêu cầu.
5. Ghi rõ những trường hợp chưa đủ căn cứ và cần người kiểm thử xác minh thêm. Không được xem mọi test FAIL là một bug riêng. Nhiều test case có thể cùng phản ánh một lỗi gốc. Ngược lại, không gộp các lỗi khác nhau chỉ vì chúng xuất hiện trong cùng một feature.

Đối với FR-11:

- Kiểm tra TC03 và TC10. Nếu cả hai cùng phản ánh vấn đề trạng thái đơn hàng không được phân biệt bằng màu sắc, hãy liên kết chúng với cùng một bug report.
- TC04 và TC05–TC08 đang BLOCKED không được ghi thành bug ứng dụng chỉ vì chưa đủ dữ liệu thực hiện kiểm thử. Đối với FR-01, FR-09 và FR-17, hãy sử dụng kết quả thực tế cùng evidence hiện có. Không tự suy luận kết quả từ tên test case hoặc chỉ dựa vào exit code.

3. Tạo hoặc cập nhật Bug Report

Tận dụng cấu trúc `bug-reports/` hiện có. Nếu chưa có quy ước, tạo một file tổng hợp `bug-reports/README.md` và các file bug riêng khi cần.

Mỗi bug report đã xác minh phải có:

- Bug ID duy nhất.
- Tiêu đề ngắn gọn, mô tả đúng vấn đề.
- Feature và các test case liên quan.
- Mức độ nghiêm trọng nếu có đủ căn cứ; nếu chưa thì ghi chưa đánh giá.
- Môi trường kiểm thử.
- Tiền điều kiện.
- Các bước tái hiện cụ thể.
- Expected result dựa trên SRS.
- Actual result dựa trên kiểm thử thực tế.
- Tần suất tái hiện nếu đã được kiểm tra.
- Screenshot và log thực tế liên quan.
- Trạng thái xác minh và ghi chú cần điều tra thêm. Mỗi bug report phải trỏ tới screenshot thực tế có thể mở được. Nếu chưa có ảnh phù hợp, ghi rõ thiếu evidence và không giả vờ rằng ảnh đã tồn tại.

4. Bảo toàn kết quả hiện có

- Không sửa test plan gốc.
- Không thay đổi expected result.
- Không đổi FAIL thành PASS hoặc BLOCKED nếu chưa có căn cứ.
- Không reset database, sửa source code hoặc chạy lại toàn bộ test chỉ để làm đẹp báo cáo.
- Không xóa evidence cũ.
- Không tự tạo screenshot, log hoặc bug chưa được xác minh.
- Không commit hoặc push Git.

5. Chuẩn bị cho GitHub Issues

Tạo nội dung issue bằng Markdown để có thể đăng lên GitHub sau khi tôi rà soát. Mỗi issue cần có tiêu đề, mô tả, bước tái hiện, expected result, actual result và đường dẫn screenshot.

Chưa đăng hoặc tạo issue trực tuyến trong nhiệm vụ này. Trước hết hãy hoàn thiện tài liệu local, tránh tạo issue trùng lặp với những issue đã tồn tại nếu có thể kiểm tra được.

6. Kiểm tra cuối cùng

Sau khi hoàn thành:

- Đối chiếu từng bug với test case và evidence.
- Thống kê số bug đã xác minh, số trường hợp cần điều tra thêm và số test case liên quan.
- Đảm bảo mỗi bug report không bị trùng với bug khác.
- Liệt kê các file đã tạo hoặc cập nhật.
- Nêu rõ các issue cần tôi kiểm tra và đăng lên GitHub. Hãy thực sự rà soát workspace và cập nhật các file Bug Report. Không chỉ đưa ra kế hoạch hoặc viết ví dụ chung chung.

- **Tóm tắt đầu ra AI:** Đọc và đối chiếu test plan, execution results, SRS, evidence index, logs/traces/screenshots; nhóm các kết quả FAIL theo hành vi/nguyên nhân và tách các vấn đề độc lập. Ghi nhận 10 lỗi đã xác minh, 27 test case FAIL liên quan, 7 ca cần dữ liệu/thực thi thêm (FR-01 TC11/TC12 NOT AUTOMATED; FR-11 TC04–TC08 BLOCKED). Tìm issue trực tuyến theo nhóm từ khóa, không tạo issue.
- **Tệp và thao tác:** Tạo `bug-reports/README.md` và 10 báo cáo `BR-FR01-01.md`–`BR-FR01-03.md`, `BR-FR09-01.md`–`BR-FR09-03.md`, `BR-FR17-01.md`–`BR-FR17-03.md`, `BR-FR11-01.md`. Không chạy lại test, không sửa nguồn, không commit/push.
- **Kết quả thực tế:** Các file báo cáo được tạo. Phần rà soát cuối xác nhận liên kết local và ảnh được dẫn tồn tại. Không tạo issue trực tuyến.

- **Rà soát của người dùng:** Prompt yêu cầu giữ nguyên kết quả và evidence, không biến BLOCKED thành bug, không tạo screenshot giả; những giới hạn này được giữ trong báo cáo.
- **Giới hạn/chưa xác minh:** Không có rubric severity nên severity được ghi chưa đánh giá. Một số ca API-only không có screenshot; báo cáo nêu rõ thiếu ảnh và dùng JSON/trace có sẵn. Tìm kiếm issue không chứng minh tuyệt đối không tồn tại issue tương tự ngoài truy vấn đã dùng.

COPILOT-04 — Việt hóa Bug Reports theo góp ý người dùng

- **Thời gian:** 16/10/2026 21:55:18 (UTC+7), theo timestamp trong hội thoại.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> ghi bằng tiếng việt cho tôi sao lại ghi tiếng anh

- **Tóm tắt đầu ra AI:** Xin lỗi vì đã viết báo cáo bằng tiếng Anh; Việt hóa README tổng hợp và cả 10 báo cáo, giữ nguyên ID, tên trường/API, actual result và liên kết evidence.
- **Tệp và thao tác:** Cập nhật `bug-reports/README.md` và 10 file `bug-reports/BR-*.md`; kiểm tra lại Markdown links.
- **Kết quả thực tế:** Tìm kiếm các heading/nhãn tiếng Anh của cấu trúc báo cáo không còn kết quả; tất cả liên kết local trong 11 file report trỏ tới tệp có thật.
- **Rà soát của người dùng:** Người dùng yêu cầu nội dung bằng tiếng Việt. Các giá trị API/ID và nội dung response thực tế được giữ nguyên để không làm sai evidence.
- **Giới hạn/chưa xác minh:** Tên mã, identifier, HTTP status, tên endpoint và response nguyên văn vẫn được giữ theo kỹ thuật/nguồn gốc; không dịch nội dung literal của ứng dụng.

COPILOT-05 — AI Gap Analysis cho FR-01, FR-09, FR-17, FR-11

- **Thời gian:** 16/10/2026 22:19:22 (UTC+7), theo timestamp trong hội thoại.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # NHIỆM VỤ: THỰC HIỆN AI GAP ANALYSIS CHO HW02
>
> Hãy thực hiện phân tích khoảng trống của AI (AI Gap Analysis) cho bốn feature FR-01, FR-09, FR-17 và FR-11 trong bài HW02.
>
> Mục tiêu là xác định những test case hoặc bug mà AI ban đầu bỏ sót, đề xuất chưa đầy đủ, đánh giá sai hoặc chưa phát hiện được; giải thích nguyên nhân và ghi nhận vai trò kiểm tra, điều chỉnh của con người.
>
> Không tạo nội dung chung chung chỉ để hoàn thành tài liệu. Mọi nhận định phải dựa trên tài liệu, prompt, đầu ra AI, kết quả kiểm thử hoặc evidence thực sự có thể kiểm tra.
>
> ## 1. Khảo sát tài liệu
>
> Tìm và đọc các nguồn sau trong workspace:
>
> - Đặc tả SRS gốc của EShop.
> - Domain Testing và BVA của FR-01, FR-09, FR-17 và FR-11.
> - Các file execution-results.md tương ứng.
> - evidence-index.md và evidence thực tế.
> - Các bug report đã được lập.
> - Tài liệu Agent Skill: SKILL.md, FR-01-domain-bva-analysis.md và FR-01-skill-validation.md.
> - Các prompt hoặc bản đầu ra AI trước đây nếu còn được lưu trong repo.
> - AI Audit hoặc AI Critique hiện có nếu đã được tạo. Phân biệt rõ:
>
> 1. Đặc tả nghiệp vụ gốc.
> 2. Đầu ra test design ban đầu của AI.
> 3. Các chỉnh sửa hoặc phân tích bổ sung của con người.
> 4. Kết quả kiểm thử thực tế. Nếu thiếu prompt hoặc đầu ra AI ban đầu, hãy ghi rõ giới hạn đó. Không tự sáng tác lịch sử tương tác AI và không khẳng định AI đã bỏ sót một test case nếu không có căn cứ đối chiếu.
>
> ## 2. Phân tích từng feature
>
> Với từng FR-01, FR-09, FR-17 và FR-11, hãy trả lời:
>
> - AI ban đầu đã thiết kế những nhóm test case nào?
> - Có yêu cầu hoặc lớp tương đương nào chưa được bao phủ?
> - Có giá trị biên nào có căn cứ từ SRS nhưng chưa được thiết kế hoặc diễn giải đúng không?

- AI có đặt expected result vượt quá những gì đặc tả quy định không?
- Có trường hợp nào AI tạo test case trùng lặp hoặc phân loại chưa hợp lý không?
- Kiểm thử thực tế có phát hiện vấn đề mà test design ban đầu không dự đoán hoặc chưa kiểm tra đầy đủ không?
- Con người đã phát hiện, điều chỉnh hoặc bổ sung điều gì?
- Có khoảng trống nào vẫn chưa thể kiểm tra do thiếu dữ liệu, môi trường hoặc giới hạn công cụ không? Đối chiếu kỹ các vấn đề đã biết nhưng không được mặc định rằng chúng đều là lỗi của AI:
- FR-01: kiểm tra email, mật khẩu, trường xác nhận mật khẩu và các test FAIL thực tế. Phân biệt lỗi ứng dụng với thiếu sót của test design.
- FR-09 và FR-17: dựa trên Domain Testing, BVA, kết quả chạy và bug report thật để xác định mức độ bao phủ.
- FR-11: TC03 và TC10 ghi nhận vấn đề thiếu màu sắc phân biệt trạng thái; TC04 và TC05–TC08 bị BLOCKED do thiếu điều kiện dữ liệu hoặc không thể tạo dữ liệu thử nghiệm an toàn.
- Skill Domain Testing/BVA: ghi nhận các điểm chưa đúng hoặc chưa đầy đủ được phát hiện khi đối chiếu với SRS FR-01, nhưng không xem bản demo Skill mới là bằng chứng rằng mọi thiếu sót trước đó đều do AI gây ra.

3. Phân loại khoảng trống

Mỗi phát hiện phải thuộc một nhóm phù hợp, chẳng hạn:

- Missing test coverage: thiếu test case hoặc lớp tương đương.
- Missing boundary coverage: bỏ sót biên có căn cứ trong đặc tả.
- Incorrect assumption: giả định hoặc expected result chưa có căn cứ.
- Missing defect detection: lỗi ứng dụng không được test design ban đầu dự đoán.
- Testability/environment gap: chưa kiểm thử được do môi trường hoặc dữ liệu.
- Human correction: điểm con người đã phát hiện hoặc sửa trong quá trình rà soát AI. Không ép phải có đủ phát hiện trong mọi nhóm. Nếu chưa có bằng chứng về một nhóm, ghi rõ chưa xác định được.

Với mỗi phát hiện, ghi:

- FR và test case liên quan.
- Yêu cầu nguồn làm căn cứ.
- AI output ban đầu, nếu có.
- Khoảng trống hoặc sai lệch được phát hiện.
- Bằng chứng thực tế.
- Vai trò của con người.
- Ảnh hưởng đối với chất lượng kiểm thử.
- Trạng thái: đã xử lý, chưa xử lý hoặc chưa đủ thông tin kết luận.

4. Tạo báo cáo AI Gap Analysis

Tạo file mới:

`ai-gap-analysis.md`

Nếu repo có quy ước hoặc vị trí lưu báo cáo AI Gap Analysis khác, hãy tuân theo cấu trúc hiện tại và ghi rõ đường dẫn.

Nội dung gồm:

1. Mục tiêu và phạm vi.
2. Nguồn dữ liệu dùng để phân tích.
3. Phương pháp đối chiếu AI output với SRS, test design và actual test results.
4. Bảng tổng hợp phát hiện theo từng FR.
5. Chi tiết các gap đáng chú ý và bằng chứng.
6. Các điểm con người đã phát hiện hoặc điều chỉnh.
7. Các khoảng trống chưa xử lý và hạn chế của phân tích.
8. Kết luận về chất lượng đầu ra AI và cách con người kiểm soát rủi ro.

Tạo bảng với các cột:

`Gap IDFRCategoryAI Output / Initial CoverageGap FoundEvidence / RequirementHuman ActionStatus`

Không tự tạo thống kê hoặc kết quả thử nghiệm. Nếu không thể xác minh số lượng test case ban đầu, hãy ghi rõ chưa xác định được.

5. Quy tắc bảo toàn dữ liệu

> - Không thay đổi source code EShop.
> - Không sửa các file Domain Testing, BVA hoặc execution results gốc trong nhiệm vụ này.
> - Không thay đổi trạng thái PASS/FAIL/BLOCKED.
> - Không sửa expected result để phù hợp với actual result.
> - Không chạy lại các test đã hoàn thành chỉ để phục vụ việc viết Gap Analysis.
> - Không reset database, xóa evidence hoặc tạo bug report trùng lặp.
> - Không tự đăng GitHub Issues.
> - Không commit hoặc push Git. Chỉ tạo hoặc cập nhật tài liệu AI Gap Analysis cần thiết.
>
> ## 6. Kiểm tra cuối cùng
>
> Sau khi viết:
>
> - Kiểm tra mỗi phát hiện có căn cứ và đường dẫn hợp lệ.
> - Đảm bảo lỗi ứng dụng, khoảng trống test design và test bị BLOCKED không bị đánh đồng.
> - Kiểm tra không có tuyên bố AI đã bỏ sót hoặc phát hiện lỗi mà không có bằng chứng.
> - Báo cáo các file đã tạo hoặc thay đổi và những điểm cần tôi xác minh. Hãy bắt đầu bằng việc khảo sát các nguồn thực tế, sau đó viết và kiểm tra `ai-gap-analysis.md`. Không chỉ đưa ra dàn ý hoặc mẫu minh họa.

- **Tóm tắt đầu ra AI:** Khảo sát SRS, test design, execution/evidence, bug reports, AI audit/critique và Skill/demo FR-01. Ghi nhận không có transcript ban đầu cho FR-17/FR-11 và `ai-critique.md` rỗng. Tạo phân tích gồm 12 gap, phân biệt các vấn đề xác minh được với attribution chưa rõ.
- **Tệp và thao tác:** Tạo `ai-gap-analysis.md`; đọc nhiều tài liệu thiết kế, kết quả, evidence index/ảnh, bug report, `ai-audit.md`, `ai-critique.md`, Skill và hai demo. Kiểm tra liên kết local và cấu trúc bảng; không chạy test.
- **Kết quả thực tế:** Báo cáo có bảng 12 dòng, tất cả liên kết nội bộ được xác minh là tồn tại. Không sửa test plan, execution, evidence, source hoặc database.
- **Rà soát của người dùng:** Prompt yêu cầu không quy kết thiếu sót cho AI nếu thiếu prompt/output ban đầu. Báo cáo ghi FR-17/FR-11 chưa đủ thông tin quy nguồn; phân biệt UI validation với API behavior và giữ FR-11 BLOCKED.
- **Giới hạn/chưa xác minh:** Không có transcript AI đầy đủ; nhật ký FR-01/FR-09 là bản tóm tắt. Không chạy lại test để kiểm tra trạng thái hiện tại.

COPILOT-06 — Cập nhật AI Audit cuối phiên

- **Thời gian:** 16/10/2026 22:30:48 (UTC+7), theo timestamp trong hội thoại.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # PROMPT AUDIT CUỐI SESSION — HW02
>
> Bạn là trợ lý ghi nhận AI Audit cho bài tập HW02 – Domain Testing. Tôi làm việc với GitHub Copilot theo nhiều session riêng biệt, có thể đóng VS Code hoặc chuyển sang session khác bất cứ lúc nào. Vì vậy, sau mỗi session, bạn phải cập nhật nhật ký AI Audit để bảo đảm quá trình sử dụng AI được ghi nhận liên tục, trung thực và có thể truy vết.
>
> ## 1. Xác định file AI Audit
>
> Tìm file `ai-audit.md` trong repo HW02 hiện tại.
>
> Nếu chưa có file, hãy tạo file tại vị trí phù hợp trong repo HW02. Nếu đã tồn tại, hãy đọc nội dung hiện tại trước khi cập nhật.
>
> Nếu workspace đang mở cả repo EShop và HW02, hãy xác định đúng thư mục HW02 để ghi nhật ký. Không ghi AI Audit vào repo EShop chỉ vì terminal hiện đang ở thư mục EShop.
>
> ## 2. Rà soát toàn bộ session hiện tại
>
> Dựa trên lịch sử hội thoại Copilot còn truy cập được trong session này, các file được đọc hoặc sửa, kết quả lệnh, kết quả chạy test và các hành động thực tế, hãy xác định tất cả tương tác AI có ý nghĩa liên quan đến bài HW02.
>
> Ghi nhận những công việc đã thực sự thực hiện, chẳng hạn:
>
> - Phân tích yêu cầu và thiết kế Domain Testing.
> - Thiết kế Boundary Value Analysis (BVA).
> - Tạo hoặc chỉnh sửa test case.
> - Viết, chạy hoặc sửa script kiểm thử tự động.
> - Kiểm thử thủ công và phân tích kết quả.
> - Thu thập, sắp xếp screenshot, log và evidence.
> - Phân tích bug và tạo bug report.
> - Thực hiện AI Gap Analysis.

- Tạo, kiểm tra hoặc áp dụng Agent Skill.
- Soạn thảo, rà soát hoặc hoàn thiện báo cáo. Không bắt buộc phải ghi các thao tác thông thường không có ý nghĩa đối với việc sử dụng AI trong bài tập. Tuy nhiên, không bỏ sót các tương tác AI quan trọng chỉ vì chúng diễn ra ở nhiều lượt chat khác nhau.

3. Ghi lại thông tin cho từng tương tác AI

Với mỗi tương tác có ý nghĩa, ghi các thông tin sau:

1. **Interaction ID:** mã duy nhất, tăng dần hoặc theo quy ước hiện có.
2. **AI tool:** tên công cụ, ví dụ GitHub Copilot.
3. **Date and time:** ngày giờ thực tế nếu xác định được; sử dụng múi giờ Việt Nam (UTC+7) khi phù hợp.
4. **Context/Task:** mục tiêu công việc.
5. **User prompt:** prompt thực tế đã gửi cho AI.
6. **AI output:** đầu ra thực tế của AI, có thể tóm tắt nếu đầu ra quá dài nhưng phải giữ các kết luận quan trọng và những thay đổi được đề xuất.
7. **Tệp và thao tác:** các file đã đọc, tạo, sửa hoặc xóa; lệnh đã chạy; test đã thực thi nếu có.
8. **Actual result:** kết quả công việc thực tế, phân biệt rõ giữa đề xuất, thay đổi đã thực hiện và hành động đã xác minh.
9. **Rà soát của người dùng:** những điểm tôi đã tự kiểm tra, sửa, chấp nhận hoặc chưa xác minh.
10. **Limitations:** các giới hạn, lỗi, thông tin thiếu hoặc việc chưa hoàn thành.
11. **Related artifacts:** đường dẫn tới test plan, execution results, evidence, bug report hoặc tài liệu liên quan nếu có.

Quy tắc về prompt và đầu ra

- Nếu lịch sử chat hiện tại còn chứa prompt và đầu ra gốc, hãy ghi lại chính xác prompt đó; với đầu ra dài, ghi tóm tắt trung thực và dẫn tới file kết quả liên quan.
- Nếu lịch sử cũ không còn truy cập được, không tự dựng lại prompt hoặc câu trả lời như thể đó là nguyên văn. Ghi rõ `Prompt gốc không còn truy cập được` hoặc `Chỉ ghi nhận được bản tóm tắt`.
- Nếu không xác định được ngày giờ chính xác, ghi `Không xác định được`; không tự tạo timestamp.
- Nếu một hành động chỉ được đề xuất nhưng chưa được thực hiện, phải ghi là `Đề xuất`, không phải `Đã thực hiện`.
- Nếu script đã chạy nhưng có test FAIL, giữ đúng trạng thái FAIL.
- Nếu test bị BLOCKED hoặc NOT AUTOMATED, không ghi là đã hoàn thành.
- Không khẳng định đã đọc, sửa, chạy, kiểm tra hoặc xác minh một file nếu không có căn cứ thực tế.

4. Cập nhật AI Audit theo kiểu nối tiếp

Trong `ai-audit.md`:

- Giữ nguyên các nội dung hợp lệ đã được ghi từ session trước.
- Chỉ bổ sung các tương tác mới của session hiện tại.
- Không ghi đè toàn bộ file cũ.
- Không tự xóa các mục cũ vì cho rằng chúng trùng lặp hoặc không quan trọng.
- Kiểm tra Interaction ID để tránh tạo hai mục giống nhau.
- Nếu phát hiện một mục cũ sai hoặc cần sửa, ghi rõ nội dung nào được sửa và lý do; không âm thầm thay đổi lịch sử.
- Giữ cấu trúc dễ đọc bằng Markdown và sử dụng cùng quy ước trình bày trong toàn bộ file. Có thể dùng bảng tổng quan với các cột:

```
Interaction ID | Date/time | AI tool | Task | Result | Human review | Artifacts
```

Sau bảng tổng quan, bổ sung phần chi tiết cho từng tương tác, gồm prompt và đầu ra AI theo khả năng truy cập thực tế.

Bảo đảm AI Audit đáp ứng yêu cầu HW02: khai báo công cụ AI, thời gian, prompt và đầu ra cho các tương tác có thể xác minh. Không thay thế toàn bộ lịch sử bằng một đoạn tổng kết chung chung.

5.

- **Tóm tắt đầu ra AI:** Đọc nội dung `ai-audit.md` trước khi cập nhật; nối thêm sáu mục COPILOT-01 đến COPILOT-06, trong đó các prompt dài được giữ nguyên ở phần chi tiết khi còn hiện diện trong lịch sử. Không ghi lại nội dung audit 14/10 hoặc sửa lịch sử cũ.
- **Tệp và thao tác:** Chỉ cập nhật `ai-audit.md`; đọc trước đó file audit hiện tại và kiểm tra trạng thái Git của file audit cùng các artifacts liên quan.
- **Kết quả thực tế:** Phần lịch sử cũ được giữ nguyên; nhật ký nối tiếp ghi nhận các hoạt động Skill, bug reports, Việt hóa, AI Gap Analysis và audit.
- **Rà soát của người dùng:** Yêu cầu không dựng lại prompt/timestamp thiếu căn cứ. Các giờ không hiện diện trong lịch sử được đánh dấu không xác định; các timestamp có trong user message được ghi theo UTC+7.
- **Giới hạn/chưa xác minh:** Một số lượt trước không có timestamp chính xác trong lịch sử khả dụng. Các prompt COPILOT-01/02/03/05 được ghi theo nội dung user message hiện còn truy cập; đầu ra AI dài được tóm tắt thay vì chép nguyên văn.

COPILOT-07 — Tạo cấu trúc thư mục evidence cho FR-11

- **Thời gian:** 16/10/2026 20:24:49 (UTC+7), theo timestamp trong user message.

- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> đọc file FR-11-manual-execution-results để biết cách tạo thư mục tạo thư mục evidence cho fr11 tạo xong tui sẽ dán ản screen shot vào

- **Tóm tắt đầu ra AI:** Đọc hướng dẫn evidence trong kết quả kiểm thử FR-11 và đối chiếu cấu trúc evidence hiện có. Xác định root cần tạo là `evidence/FR-11-mobile/` với các thư mục `TC01`–`TC10`; không tạo ảnh hoặc nội dung evidence giả.
- **Tệp và thao tác:** Đọc `mobile-FR-11/FR-11-manual-execution-results.md`, `evidence/evidence-index.md` và tài liệu execution FR-09 để tham khảo quy ước; kiểm tra các thư mục evidence hiện có và `git status`. Tạo `evidence/FR-11-mobile/TC01/` đến `TC10/` bằng PowerShell và liệt kê xác nhận tên thư mục.
- **Kết quả thực tế:** Mười thư mục TC01–TC10 được tạo. Lúc kiểm tra, root FR-11 chưa có ảnh; người dùng dự định bổ sung ảnh sau.
- **Rà soát của người dùng:** Người dùng cung cấp yêu cầu và sau đó bổ sung ảnh thật. Ảnh không được AI tạo.
- **Giới hạn/chưa xác minh:** Chỉ tạo cấu trúc thư mục; chưa có screenshot lúc hoàn tất tương tác này.
- **Tài liệu liên quan:** `mobile-FR-11/FR-11-manual-execution-results.md`; `evidence/FR-11-mobile/`.

COPILOT-08 — Bổ sung đường dẫn ảnh thực tế cho FR-11

- **Thời gian:** 16/10/2026 20:29:23 (UTC+7), theo timestamp trong user message.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> tui đã bổ sung ảnh, bổ sung đường dẫn ảnh thực tế. vào fle FR-11-manual-execution-results

- **Tóm tắt đầu ra AI:** Liệt kê ảnh thực tế đã được thêm, thay placeholder Evidence cho TC01, TC02, TC03, TC09 và TC10 bằng liên kết file tương đối; giữ TC04–TC08 là chưa có ảnh do trạng thái BLOCKED. Cập nhật phần tổng kết, bug evidence và sơ đồ thư mục.
- **Tệp và thao tác:** Đọc `mobile-FR-11/FR-11-manual-execution-results.md` và liệt kê file trong `evidence/FR-11-mobile/`. Cập nhật `mobile-FR-11/FR-11-manual-execution-results.md`. Chạy kiểm tra tồn tại cho năm đường dẫn ảnh và `git diff --check`.
- **Kết quả thực tế:** Xác nhận tồn tại `TC01/ev1.jpg`, `TC02/ev2.jpg`, `TC03/ev3_9_10.jpg`, `TC09/ev3_9_10.jpg`, `TC10/ev3_9_10.jpg`; các placeholder tương ứng được thay bằng liên kết.
- **Rà soát của người dùng:** Ảnh được người dùng bổ sung; AI chỉ đọc tên/path và liên kết vào tài liệu, không tự tạo hay sửa ảnh. TC03, TC09 và TC10 tham chiếu cùng tên ảnh trong các thư mục case riêng.
- **Giới hạn/chưa xác minh:** Không xem xét nội dung hình ảnh để đánh giá lại kết quả kiểm thử; chỉ xác minh đường dẫn tồn tại. TC04–TC08 vẫn BLOCKED và không có evidence thực thi.
- **Tài liệu liên quan:** `mobile-FR-11/FR-11-manual-execution-results.md`; `evidence/FR-11-mobile/TC01/ev1.jpg`; `evidence/FR-11-mobile/TC02/ev2.jpg`; `evidence/FR-11-mobile/TC03/ev3_9_10.jpg`; `evidence/FR-11-mobile/TC09/ev3_9_10.jpg`; `evidence/FR-11-mobile/TC10/ev3_9_10.jpg`.

COPILOT-09 — Tạo Skill Domain Testing/BVA và demo FR-01

- **Thời gian:** 16/10/2026 20:40:44 (UTC+7), theo timestamp trong user message.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # NHIỆM VỤ: TẠO AGENT SKILL TÁI SỬ DỤNG DOMAIN TESTING VÀ BVA CHO HW02
>
> Hãy tạo một Agent Skill thực sự có thể tái sử dụng trong GitHub Copilot Agent mode để hỗ trợ kỹ thuật Domain Testing và Boundary Value Analysis (BVA) cho dự án EShop.
>
> Đây là phần bổ sung còn thiếu của HW02. Các test case, kết quả kiểm thử và evidence của FR-01, FR-09, FR-17 và FR-11 đã được thực hiện trước đó. Không xây dựng lại những phần này.
>
> ## 1. Tạo Skill đúng cấu trúc
>
> Tạo Skill tại:
>
> `.github/skills/eshop-domain-bva-testing/SKILL.md`
>
> File phải có YAML frontmatter hợp lệ, gồm:
>
> - `name: eshop-domain-bva-testing`
> - `description`: mô tả rõ khả năng và thời điểm sử dụng Skill. Nội dung SKILL.md phải là hướng dẫn có thể dùng lại cho nhiều feature, không chỉ chứa test case cố định của FR-01.
>
> Tạo thêm tài liệu hỗ trợ trong thư mục `agent-skills/` để phục vụ việc nộp HW02. Nếu cần sao chép nội dung Skill vào thư mục này, phải bảo đảm bản sao đồng nhất với file Skill thực tế, không để hai phiên bản khác nhau.

2. Nội dung Skill phải hướng dẫn quy trình Domain Testing

Khi nhận đặc tả một feature, Skill phải hướng dẫn Copilot thực hiện theo các bước:

1. Đọc đặc tả nghiệp vụ và xác định đúng phạm vi feature.
2. Trích xuất yêu cầu, điều kiện đầu vào, biến đầu vào, biến đầu ra và hành vi mong đợi.
3. Xác định các điều kiện có thể ảnh hưởng đến kết quả xử lý.
4. Phân chia các lớp tương đương hợp lệ và không hợp lệ.
5. Giải thích căn cứ của từng lớp tương đương dựa trên yêu cầu cụ thể.
6. Chọn test case đại diện để bao phủ các lớp đã xác định.
7. Với trường hợp invalid, cố gắng kiểm tra từng lớp invalid riêng biệt và giữ các điều kiện khác hợp lệ.
8. Ghi mã test case, tiền điều kiện, dữ liệu kiểm thử, các bước thực hiện và expected result.
9. Kiểm tra độ bao phủ: mỗi lớp tương đương phải được ánh xạ tới ít nhất một test case hoặc được giải thích rõ nếu chưa thể kiểm thử.
10. Rà soát lại các test case để tìm lớp bị bỏ sót, trường hợp trùng lặp hoặc expected result chưa có căn cứ.

3. Nội dung Skill phải hướng dẫn BVA

Skill phải thực hiện quy trình BVA riêng biệt:

1. Tìm các miền dữ liệu có biên được quy định trong đặc tả.
2. Xác định biên dưới, biên trên hoặc các giới hạn tương ứng.
3. Thiết kế các trường hợp ngay dưới biên, tại biên và ngay trên biên khi phù hợp với kỹ thuật và yêu cầu.
4. Giữ các yếu tố khác hợp lệ nhằm cô lập ảnh hưởng của biên đang kiểm tra.
5. Ghi test case ID, miền dữ liệu, giá trị kiểm thử, expected result và căn cứ của biên.
6. Đối chiếu từng test case BVA với yêu cầu gốc. Không được tự bịa giới hạn. Nếu đặc tả không quy định biên, Skill phải nêu rõ chưa đủ căn cứ để thiết kế BVA chính thức cho miền dữ liệu đó; không ép mọi feature phải có các giá trị biên giả định.

Phải phân biệt Domain Testing với BVA và không nhầm các lớp tương đương rời rạc với các giá trị biên số học.

4. Quy tắc chất lượng bắt buộc

Skill phải yêu cầu Copilot:

- Dẫn ra yêu cầu hoặc câu trong đặc tả làm căn cứ cho mỗi điều kiện và expected result.
- Ghi rõ giả định khi cần thiết và không trình bày giả định như quy tắc nghiệp vụ.
- Không tự tạo ra yêu cầu nghiệp vụ chưa được đặc tả.
- Phân biệt thông tin chưa xác định với dữ liệu invalid.
- Không chỉ tạo bảng test case mà phải giải thích các bước áp dụng kỹ thuật.
- Tạo bảng có thể đối chiếu giữa yêu cầu, lớp tương đương, biên và test case.
- Kiểm tra mã test case và tính nhất quán của kết quả.
- Đưa ra câu hỏi hoặc ghi chú khi thiếu dữ liệu thay vì tự suy đoán. Kết quả phải phù hợp với tài liệu HW02, trình bày bằng tiếng Việt và có thể lưu thành Markdown.

5. Tạo tài liệu hỗ trợ nộp bài

Tạo hoặc cập nhật:

`agent-skills/README.md`

Tài liệu này phải trình bày:

- Tên Skill và mục tiêu.
- Cấu trúc file.
- Khi nào sử dụng Skill.
- Cách gọi Skill trong Copilot Agent mode.
- Đầu vào mà người dùng cần cung cấp.
- Các bước xử lý của Skill.
- Đầu ra mong đợi.
- Cách kiểm tra chất lượng đầu ra.
- Giới hạn của Skill. Tạo:

`agent-skills/demo-plan.md`

Nội dung gồm kịch bản demo từ đầu đến cuối, sử dụng FR-01 Registration làm ví dụ:

1. Mở đặc tả FR-01.
2. Gọi Skill `eshop-domain-bva-testing`.
3. Yêu cầu Skill phân tích Domain Testing.

4. Yêu cầu Skill phân tích BVA cho điều kiện mật khẩu tối thiểu 8 ký tự nếu điều kiện đó được xác nhận trong đặc tả.
5. Quan sát đầu ra.
6. Đối chiếu kết quả với đặc tả và chỉnh sửa các điểm chưa hợp lý nếu có.
7. Tổng kết điều Skill làm tốt và giới hạn còn tồn tại. Không tuyên bố Skill đã được gọi hoặc áp dụng thành công nếu chưa thực sự thử nghiệm.

6. Thực sự kiểm tra Skill

Sau khi tạo Skill:

1. Kiểm tra cú pháp YAML frontmatter và cấu trúc thư mục.
2. Kiểm tra Copilot có thể phát hiện Skill theo cơ chế hỗ trợ của workspace hay không.
3. Dùng Skill để thực hiện một lượt phân tích FR-01, tạo đầu ra minh họa mới trong `agent-skills/demo/`.
4. Đầu ra demo phải thể hiện được cả Domain Testing và BVA.
5. Đối chiếu đầu ra demo với tài liệu FR-01 đang có; không ghi đè hoặc thay đổi các file test plan, execution results hay evidence hiện có.
6. Ghi nhận các điểm cần con người rà soát và các giới hạn nếu có. Nếu Skill không được Copilot tự động nạp, kiểm tra nguyên nhân và hướng dẫn cách gọi trực tiếp trong Agent mode. Không chỉ tạo file rồi mặc định là Skill hoạt động.

7. Chuẩn bị video demo

Không cần tự quay hoặc tải video lên YouTube thay tôi.

Hãy chuẩn bị kịch bản dễ thực hiện trực tiếp trên VS Code, cho thấy quá trình sử dụng Skill từ đầu vào đến đầu ra trên FR-01. Video cần thể hiện:

- Skill và cách gọi.
- Đặc tả được đưa vào.
- Các bước Domain Testing.
- Các bước BVA.
- Đầu ra do Skill tạo.
- Việc con người đối chiếu, phát hiện hoặc chỉnh sửa kết quả AI. Cuối cùng, báo cáo danh sách file được tạo, kết quả kiểm tra Skill, đường dẫn đầu ra demo và các bước tôi cần thực hiện để tự quay video.

Không commit hoặc push Git. Không sửa ứng dụng, database, execution results hoặc evidence của các FR đã hoàn thành.

Bắt đầu bằng việc kiểm tra cấu trúc workspace hiện tại, sau đó tạo Skill và thử áp dụng thực tế.

- **Tóm tắt đầu ra AI:** Đọc hướng dẫn cấu trúc Skill, khảo sát workspace, `agent-skills/` và tài liệu FR-01 hiện có. Tạo hướng dẫn Skill đa feature cho Domain Testing/BVA, README nộp bài, kịch bản demo/video và bản phân tích minh họa FR-01 với EC01–EC19, TC01–TC12, BVA-FR01-01–03. Trong demo, đánh dấu điều kiện mật khẩu tối thiểu 8 ký tự và các điều kiện lấy từ test plan là cần xác nhận vì không tìm thấy SRS FR-01 gốc trong repo. Kết quả thiết kế không ghi actual result hay trạng thái thực thi.
- **Tệp và thao tác:** Đọc tài liệu quy ước Skill của VS Code và các tài liệu FR-01 Domain Testing/BVA; kiểm tra `.github/` chưa có và `agent-skills/` ban đầu chỉ có `.gitkeep`; tìm kiếm tài liệu SRS/specification; tạo `.github/skills/eshop-domain-bva-testing/SKILL.md`, `agent-skills/README.md`, `agent-skills/demo-plan.md`, `agent-skills/demo/FR-01-domain-bva-analysis.md`. Chạy Python/PyYAML kiểm tra frontmatter, tên thư mục, kích thước Skill, liên kết Markdown và các ID demo; chạy `git diff --check`.
- **Kết quả thực tế:** Các file được tạo; kiểm tra YAML, liên kết và ID cuối cùng đạt. Không sửa test plan, execution results, evidence, ứng dụng hay database. Có một lượt kiểm tra trung gian bị lỗi do kiểm tra script (TypeError/kiểm tra chuỗi); sau đó điều kiện kiểm tra được chỉnh và lượt xác thực cuối thành công.
- **Rà soát của người dùng:** Người dùng yêu cầu không giả định Skill đã được gọi. AI ghi rõ chưa có khả năng điều khiển Copilot Agent mode/slash picker trong phiên, nên không xác minh được Copilot tự phát hiện/nạp Skill. Bản demo được soạn theo workflow mới nhưng không được trình bày là bằng chứng Copilot Agent đã thực thi Skill. Cần con người đối chiếu điều kiện nghiệp vụ và expected result với SRS chính thức.
- **Giới hạn/chưa xác minh:** Không có SRS FR-01 gốc trong workspace để xác nhận các điều kiện đang ghi ở Domain Testing/BVA. Không thể thử slash invocation/automatic discovery thực tế trong UI Agent mode ở phiên này; README và demo-plan hướng dẫn người dùng tự kiểm tra. Không chạy test ứng dụng và không cập nhật các kết quả/evidence cũ.
- **Tài liệu liên quan:** `.github/skills/eshop-domain-bva-testing/SKILL.md`; `agent-skills/README.md`; `agent-skills/demo-plan.md`; `agent-skills/demo/FR-01-domain-bva-analysis.md`; `FR-01-registration/domain-testing.md`; `FR-01-registration/bva.md`.

COPILOT-10 — Audit cuối session và nối nhật ký AI Audit

- **Thời gian:** 16/10/2026 22:45:26 (UTC+7), theo timestamp trong user message.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # PROMPT AUDIT CUỐI SESSION — HW02

Bạn là trợ lý ghi nhận AI Audit cho bài tập HW02 – Domain Testing. Tôi làm việc với GitHub Copilot theo nhiều session riêng biệt, có thể đóng VS Code hoặc chuyển sang session khác bất cứ lúc nào. Vì vậy, sau mỗi session, bạn phải cập nhật nhật ký AI Audit để bảo đảm quá trình sử dụng AI được ghi nhận liên tục, trung thực và có thể truy vết.

1. Xác định file AI Audit

Tìm file `ai-audit.md` trong repo HW02 hiện tại.

Nếu chưa có file, hãy tạo file tại vị trí phù hợp trong repo HW02. Nếu đã tồn tại, hãy đọc nội dung hiện tại trước khi cập nhật.

Nếu workspace đang mở cả repo EShop và HW02, hãy xác định đúng thư mục HW02 để ghi nhật ký. Không ghi AI Audit vào repo EShop chỉ vì terminal hiện đang ở thư mục EShop.

2. Rà soát toàn bộ session hiện tại

Dựa trên lịch sử hội thoại Copilot còn truy cập được trong session này, các file được đọc hoặc sửa, kết quả lệnh, kết quả chạy test và các hành động thực tế, hãy xác định tất cả tương tác AI có ý nghĩa liên quan đến bài HW02.

Ghi nhận những công việc đã thực sự thực hiện, chẳng hạn:

- Phân tích yêu cầu và thiết kế Domain Testing.
- Thiết kế Boundary Value Analysis (BVA).
- Tạo hoặc chỉnh sửa test case.
- Viết, chạy hoặc sửa script kiểm thử tự động.
- Kiểm thử thủ công và phân tích kết quả.
- Thu thập, sắp xếp screenshot, log và evidence.
- Phân tích bug và tạo bug report.
- Thực hiện AI Gap Analysis.
- Tạo, kiểm tra hoặc áp dụng Agent Skill.
- Soạn thảo, rà soát hoặc hoàn thiện báo cáo. Không bắt buộc phải ghi các thao tác thông thường không có ý nghĩa đối với việc sử dụng AI trong bài tập. Tuy nhiên, không bỏ sót các tương tác AI quan trọng chỉ vì chúng diễn ra ở nhiều lượt chat khác nhau.

3. Ghi lại thông tin cho từng tương tác AI

Với mỗi tương tác có ý nghĩa, ghi các thông tin sau:

1. **Interaction ID:** mã duy nhất, tăng dần hoặc theo quy ước hiện có.
2. **AI tool:** tên công cụ, ví dụ GitHub Copilot.
3. **Date and time:** ngày giờ thực tế nếu xác định được; sử dụng múi giờ Việt Nam (UTC+7) khi phù hợp.
4. **Context/Task:** mục tiêu công việc.
5. **User prompt:** prompt thực tế đã gửi cho AI.
6. **AI output:** đầu ra thực tế của AI, có thể tóm tắt nếu đầu ra quá dài nhưng phải giữ các kết luận quan trọng và những thay đổi được đề xuất.
7. **Tệp và thao tác:** các file đã đọc, tạo, sửa hoặc xóa; lệnh đã chạy; test đã thực thi nếu có.
8. **Actual result:** kết quả công việc thực tế, phân biệt rõ giữa đề xuất, thay đổi đã thực hiện và hành động đã xác minh.
9. **Rà soát của người dùng:** những điểm tôi đã tự kiểm tra, sửa, chấp nhận hoặc chưa xác minh.
10. **Limitations:** các giới hạn, lỗi, thông tin thiếu hoặc việc chưa hoàn thành.
11. **Related artifacts:** đường dẫn tới test plan, execution results, evidence, bug report hoặc tài liệu liên quan nếu có.

Quy tắc về prompt và đầu ra

- Nếu lịch sử chat hiện tại còn chứa prompt và đầu ra gốc, hãy ghi lại chính xác prompt đó; với đầu ra dài, ghi tóm tắt trung thực và dẫn tới file kết quả liên quan.
- Nếu lịch sử cũ không còn truy cập được, không tự dựng lại prompt hoặc câu trả lời như thể đó là nguyên văn. Ghi rõ `Prompt gốc không còn truy cập được` hoặc `Chỉ ghi nhận được bản tóm tắt`.
- Nếu không xác định được ngày giờ chính xác, ghi `Không xác định được`; không tự tạo timestamp.
- Nếu một hành động chỉ được đề xuất nhưng chưa được thực hiện, phải ghi là `Đề xuất`, không phải `Đã thực hiện`.
- Nếu script đã chạy nhưng có test FAIL, giữ đúng trạng thái FAIL.
- Nếu test bị BLOCKED hoặc NOT AUTOMATED, không ghi là đã hoàn thành.
- Không khẳng định đã đọc, sửa, chạy, kiểm tra hoặc xác minh một file nếu không có căn cứ thực tế.

4. Cập nhật AI Audit theo kiểu nối tiếp

Trong `ai-audit.md`:

- Giữ nguyên các nội dung hợp lệ đã được ghi từ session trước.
- Chỉ bổ sung các tương tác mới của session hiện tại.

- Không ghi đè toàn bộ file cũ.
- Không tự xóa các mục cũ vì cho rằng chúng trùng lặp hoặc không quan trọng.
- Kiểm tra Interaction ID để tránh tạo hai mục giống nhau.
- Nếu phát hiện một mục cũ sai hoặc cần sửa, ghi rõ nội dung nào được sửa và lý do; không âm thầm thay đổi lịch sử.
- Giữ cấu trúc dễ đọc bằng Markdown và sử dụng cùng quy ước trình bày trong toàn bộ file. Có thể dùng bảng tổng quan với các cột:

`Interaction IDDate/timeAI toolTaskResultHuman reviewArtifacts`

Sau bảng tổng quan, bổ sung phần chi tiết cho từng tương tác, gồm prompt và đầu ra AI theo khả năng truy cập thực tế.

Bảo đảm AI Audit đáp ứng yêu cầu HW02: khai báo công cụ AI, thời gian, prompt và đầu ra cho các tương tác có thể xác minh. Không thay thế toàn bộ lịch sử bằng một đoạn tổng kết chung chung.

5. Phân biệt các công cụ AI

Tôi có sử dụng ChatGPT và GitHub Copilot trong quá trình làm HW02.

- Chỉ ghi GitHub Copilot cho tương tác thực hiện bằng Copilot.
- Chỉ ghi ChatGPT cho tương tác thực hiện bằng ChatGPT.
- Nếu không có lịch sử của một công cụ trong session hiện tại, không tự tạo tương tác cho công cụ đó.
- Nếu có bằng chứng về tương tác ở các session trước trong file audit, hãy giữ nguyên và liên kết đúng với những tương tác đó.

6. Kiểm tra sau khi cập nhật

Sau khi ghi audit:

1. Kiểm tra Markdown và các bảng không bị lỗi định dạng.
2. Kiểm tra đường dẫn đến các file liên quan nếu có thể.
3. Đảm bảo mục vừa thêm không trùng với mục trước.
4. Liệt kê các tương tác đã ghi nhận.
5. Nêu rõ những prompt, đầu ra, ngày giờ hoặc thông tin nào chưa thể xác minh để tôi bổ sung thủ công.
6. Xác nhận đường dẫn file `ai-audit.md` đã cập nhật. Không sửa source code, database, test plan, execution results hoặc evidence chỉ để phục vụ việc audit. Không reset database, không commit hoặc push Git.

Bắt đầu bằng việc đọc AI Audit hiện tại và rà soát lịch sử session đang truy cập được. Sau đó, trực tiếp cập nhật `ai-audit.md` và báo cáo kết quả. Không chỉ đưa ra mẫu hoặc hướng dẫn.

- **Tóm tắt đầu ra AI:** Đọc audit hiện có, xác định các Interaction ID tiếp theo là `COPILOT-07`–`COPILOT-10`, nối thêm các mục cho tạo evidence FR-11, cập nhật liên kết screenshot, tạo Skill/demo, và audit hiện tại. Ghi rõ các thay đổi đã kiểm tra, các giới hạn và không sửa lịch sử trước đó.
- **Tệp và thao tác:** Đọc `ai-audit.md` trước khi sửa; xác nhận các ID hiện có tới `COPILOT-06`; kiểm tra artifact FR-11 và Skill/demo; chỉ nối thêm nội dung vào `ai-audit.md`. Không chạy test ứng dụng, không sửa source/database/test plan/execution results/evidence, không commit/push.
- **Kết quả thực tế:** Đọc audit hiện có trước khi chỉnh sửa và nối thêm COPILOT-07–COPILOT-10, giữ nguyên nội dung trước đó. Kiểm tra cuối xác nhận 10 Interaction ID COPILOT-01–COPILOT-10 không trùng; các đường dẫn FR-11/Skill được dẫn ở các mục mới tồn tại; `git diff --check` không báo lỗi.
- **Rà soát của người dùng:** Các thời điểm được lấy từ timestamp hiển thị trong user messages; prompt ở ba tác vụ trước được ghi lại từ nội dung còn trong lịch sử. Lượt áp dụng Skill không được mô tả như một lần Skill được Copilot Agent nạp thật.
- **Giới hạn/chưa xác minh:** Không truy cập được giao diện Copilot Agent để xác minh discovery Skill; không xác nhận nội dung SRS FR-01 gốc. Prompt audit được chép từ user message hiện vẫn truy cập được trong session.
- **Tài liệu liên quan:** `ai-audit.md`; `mobile-FR-11/FR-11-manual-execution-results.md`; `.github/skills/eshop-domain-bva-testing/SKILL.md`; `agent-skills/`.

---

4. Nhật ký GitHub Copilot — nhóm tương tác bổ sung COPILOT-11–COPILOT-14

Các tương tác dưới đây được nối tiếp sau COPILOT-10. Prompt/đầu ra được ghi theo nội dung còn trong hội thoại và file workspace; các lượt dùng attachment được dẫn theo tên attachment, không giả làm một prompt văn bản được gõ trực tiếp. Việc chạy test được thực hiện từ workspace EShop `D:\eshop-sut`; khi audit kiểm tra lại, báo cáo thực thi và bằng chứng tương ứng hiện có trong repo HW02 `D:\HW02`. Mã automation hiện không còn tìm thấy ở `D:\eshop-sut` hoặc `D:\HW02`, nên không khẳng định các script đã được lưu bền vững cùng evidence.

Bảng tổng quan bổ sung

Mã tương tác	Ngày/giờ (UTC+7)	Công cụ AI	Nhiệm vụ	Kết quả thực tế	Rà soát của người dùng	Tài liệu liên quan
COPILOT-11	16/10/2026 16:13:57	GitHub Copilot	Giải thích vì sao một	Giải thích cấu hình Playwright cũ chỉ chụp	Người dùng tiếp tục yêu cầu tổ	`D:\eshop-sut\tests\automated\playwright.config.js`;

Mã tương tác	Ngày/giờ (UTC+7)	Công cụ AI	Nhiệm vụ	Kết quả thực tế	Rà soát của người dùng	Tài liệu liên quan
		trong VS Code	số test case không có screenshot	khi FAIL; ảnh test PASS không được tạo ở lần chạy gốc	chức evidence và bổ sung ảnh thiếu	execution `20261016-155435`
COPILOT-12	16/10/2026 16:23:53	GitHub Copilot trong VS Code	Hoàn thiện evidence cho FR-01/09/17	Theo log phiên, bổ sung capture screenshot và chạy lại 31 UI tests còn thiếu ảnh; cả 31 PASS; giữ nguyên execution lịch sử	Audit hiện tại xác nhận reports/evidence ở HW02; các test API-only có JSON/log; screenshot liên kết theo execution ID	`evidence/evidence-index.md`; ba FR `execution-results.md`; execution `20261016-evidence-1623`
COPILOT-13	16/10/2026 16:43:09	GitHub Copilot trong VS Code	Xử lý phản hồi rằng mục lục còn ghi một số case chưa có screenshot	Theo log phiên, cập nhật mục lục để phân biệt ảnh thuộc lần gốc hay lần bổ sung và liên kết riêng từng execution	Audit hiện tại xác nhận evidence index và 46 ảnh thuộc HW02; API-only/NOT AUTOMATED vẫn được phân biệt	`evidence/evidence-index.md`; ba FR `execution-results.md`
COPILOT-14	16/10/2026 22:51:21	GitHub Copilot trong VS Code	Cập nhật AI Audit cuối session vào repo HW02	Tìm đúng repo `D:\HW02`, đọc audit trước khi sửa, xác định ID tiếp theo và nối COPILOT-11–14; không sửa source/test plan/execution/evidence	Phân biệt evidence hiện có ở HW02 với mã automation không còn thấy trong hai worktree; giữ nguyên nội dung lịch sử trước	`ai-audit.md`

Chi tiết tương tác

COPILOT-11 — Giải thích screenshot bị thiếu

- **Thời gian:** 16/10/2026 16:13:57 (UTC+7), theo timestamp hiển thị trong user message.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> sao 1 số tc ko có ảnh screenshot

- **Tóm tắt đầu ra AI:** Giải thích rằng `playwright.config.js` khi đó đặt `screenshot: "only-on-failure"`, vì vậy screenshot chỉ tự tạo khi test FAIL; test PASS chỉ có JSON/log. Báo cáo 25 ảnh lịch sử được nêu là FR-01: 10, FR-09: 5, FR-17: 10. Đề xuất có thể đổi cấu hình sang `on` nếu cần screenshot cho test PASS.
- **Tệp và thao tác:** Đọc cấu hình Playwright và liệt kê screenshot lịch sử trong `D:\eshop-sut\tests\automated\test-results\executions\20261016-155435`. Không sửa file trong lượt giải thích này.
- **Kết quả thực tế:** Chỉ giải thích nguyên nhân và đề xuất cách bổ sung; chưa tạo ảnh mới ở tương tác này.
- **Rà soát của người dùng:** Người dùng cung cấp yêu cầu chi tiết tiếp theo để tổ chức evidence, bổ sung screenshot có căn cứ và bảo toàn execution lịch sử.
- **Giới hạn/chưa xác minh:** Con số ảnh phản ánh screenshot tạo tự động ở lần gốc, không khẳng định mỗi test UI đã có ảnh hợp lệ. Một số file PNG của test API-only không phải evidence giao diện phù hợp.
- **Tài liệu liên quan:** Config và execution gốc được ghi trong history là `D:\eshop-sut\tests\automated\playwright.config.js` và execution `20261016-155435`. Tại audit này, bằng chứng kiểm thử còn xác minh được trong `D:\HW02\evidence`.

COPILOT-12 — Bổ sung và tổ chức evidence FR-01/09/17

- **Thời gian:** 16/10/2026 16:23:53 (UTC+7), theo timestamp của user message có attachment.

- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng:** Đính kèm tệp chỉ đọc `Pasted text #1.txt` (257 dòng), có tiêu đề **"NHIỆM VỤ HOÀN THIỆN QA EVIDENCE CHO FR-01, FR-09 VÀ FR-17"**. Tệp yêu cầu bảo toàn lịch sử, chụp screenshot cho UI PASS/FAIL, dùng API response/JSON cho test API-only, tổ chức evidence theo testcase, đồng bộ index/report và xác minh mọi đường dẫn. Đây là bản tóm tắt nội dung tệp, không phải bản chép nguyên văn toàn bộ nội dung.
- **Tóm tắt đầu ra AI:** Sửa helper để chụp trang Web/Admin sau khi test action/assertion đã hoàn thành, thay đổi config để tránh screenshot tự động trùng lặp, thêm script tổ chức evidence và cập nhật README. Chạy lại đúng 31 UI cases thiếu ảnh: FR-01 có 3, FR-09 có 13, FR-17 có 15; cả 31 đều PASS. API-only dùng JSON/response; không tạo ảnh trống. Lần chạy gốc `20261016-155435` được giữ nguyên.
- **Tệp và thao tác:** Theo đầu ra công cụ của lượt thực hiện, test/evidence commands chạy từ `D:\eshop-sut\tests\automated`, tạo execution `20261016-evidence-1623`, cập nhật capture helper/config/README/organizer, index và reports. Audit hiện tại xác minh reports/evidence trong `D:\HW02`; không tìm thấy mã automation trong `D:\eshop-sut\tests\automated` hoặc `D:\HW02\tests\automated`.
- **Kết quả thực tế:** Log lần bổ sung ghi 31 test PASS. Index trong HW02 ghi screenshot hợp lệ cho 46 testcase: FR-01 6, FR-09 17, FR-17 23; 10 testcase API-only dùng JSON/log; FR-01 TC11/TC12 vẫn NOT AUTOMATED. Đã xác minh hiện có 46 PNG trong HW02. Lượt QA trước báo 25 ảnh lịch sử còn nguyên và DB cleanup còn 2 users, 4 coupons, 0 QA users/coupons/usage; audit này không xác minh lại hai điều kiện đó.
- **Rà soát của người dùng:** Yêu cầu của người dùng giữ nguyên expected result, FAIL lịch sử và artifacts cũ; test lại chỉ nhằm bổ sung evidence, không đổi trạng thái lịch sử. Test API-only được phân loại theo giao diện thực tế của testcase.
- **Giới hạn/chưa xác minh:** Test được chạy từ workspace EShop theo log hội thoại, nhưng mã script và execution directories không còn tìm thấy trong các workspace hiện tại. Index/report và evidence per-case hiện được tìm thấy trong HW02. Không xác định được thời điểm/cách các output được lưu vào HW02.
- **Tài liệu liên quan hiện có:** `D:\HW02\evidence\evidence-index.md`; `D:\HW02\FR-01-registration\execution-results.md`; `D:\HW02\FR-09-coupons\execution-results.md`; `D:\HW02\FR-17-coupon-management\execution-results.md`; thư mục per-case bên dưới `D:\HW02\evidence`.

COPILOT-13 — Sửa cách trình bày screenshot theo execution

- **Thời gian:** 16/10/2026 16:43:09 (UTC+7), theo timestamp hiển thị trong user message.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> ủa sao tui đọc file evidence 1 vài tc chưa có screenshot á

- **Tóm tắt đầu ra AI:** Xác định index hiển thị từng execution riêng; do đó run gốc vẫn nói "Chưa có screenshot" dù cùng testcase có ảnh ở run bổ sung. Cập nhật organizer để dòng run gốc ghi rõ ảnh được bổ sung ở execution khác và liên kết đến ảnh, đồng thời giữ dòng riêng cho execution bổ sung.
- **Tệp và thao tác:** Theo output phiên, sửa `organize-evidence.js` và chạy `npm run evidence`. Audit hiện tại đọc `D:\HW02\evidence\evidence-index.md` cùng ba FR reports; các artifacts này hiện tồn tại.
- **Kết quả thực tế:** Mục lục phân biệt screenshot thuộc run gốc hay run bổ sung; không gộp hai lần chạy thành một trạng thái. API-only có JSON; TC11/TC12 vẫn NOT AUTOMATED. Phiên trước ghi đã xác minh 195 links; organizer script hiện không còn trong workspace để chạy lại.
- **Rà soát của người dùng:** Người dùng đã nêu vấn đề sau khi xem index; nội dung được sửa để không gây hiểu nhầm rằng ảnh bổ sung hoàn toàn không tồn tại.
- **Giới hạn/chưa xác minh:** Mã organizer không tìm thấy trong workspace hiện tại; không thể chạy lại công cụ từ audit này. Không sửa reports/evidence trong lượt audit.
- **Tài liệu liên quan:** `D:\HW02\evidence\evidence-index.md`; ba FR `execution-results.md`. `organize-evidence.js` hiện không được tìm thấy.

COPILOT-14 — Audit cuối session

- **Thời gian:** 16/10/2026 22:51:21 (UTC+7), timestamp của user message audit.
- **Công cụ AI:** GitHub Copilot trong VS Code.
- **Prompt của người dùng (nguyên văn):**

> # PROMPT AUDIT CUỐI SESSION — HW02
>
> Bạn là trợ lý ghi nhận AI Audit cho bài tập HW02 – Domain Testing. Tôi làm việc với GitHub Copilot theo nhiều session riêng biệt, có thể đóng VS Code hoặc chuyển sang session khác bất cứ lúc nào. Vì vậy, sau mỗi session, bạn phải cập nhật nhật ký AI Audit để bảo đảm quá trình sử dụng AI được ghi nhận liên tục, trung thực và có thể truy vết.
>
> ## 1. Xác định file AI Audit
>
> Tìm file `ai-audit.md` trong repo HW02 hiện tại.
>
> Nếu chưa có file, hãy tạo file tại vị trí phù hợp trong repo HW02. Nếu đã tồn tại, hãy đọc nội dung hiện tại trước khi cập nhật.

Nếu workspace đang mở cả repo EShop và HW02, hãy xác định đúng thư mục HW02 để ghi nhật ký. Không ghi AI Audit vào repo EShop chỉ vì terminal hiện đang ở thư mục EShop.

2. Rà soát toàn bộ session hiện tại

Dựa trên lịch sử hội thoại Copilot còn truy cập được trong session này, các file được đọc hoặc sửa, kết quả lệnh, kết quả chạy test và các hành động thực tế, hãy xác định tất cả tương tác AI có ý nghĩa liên quan đến bài HW02.

Ghi nhận những công việc đã thực sự thực hiện, chẳng hạn:

- Phân tích yêu cầu và thiết kế Domain Testing.
- Thiết kế Boundary Value Analysis (BVA).
- Tạo hoặc chỉnh sửa test case.
- Viết, chạy hoặc sửa script kiểm thử tự động.
- Kiểm thử thủ công và phân tích kết quả.
- Thu thập, sắp xếp screenshot, log và evidence.
- Phân tích bug và tạo bug report.
- Thực hiện AI Gap Analysis.
- Tạo, kiểm tra hoặc áp dụng Agent Skill.
- Soạn thảo, rà soát hoặc hoàn thiện báo cáo. Không bắt buộc phải ghi các thao tác thông thường không có ý nghĩa đối với việc sử dụng AI trong bài tập. Tuy nhiên, không bỏ sót các tương tác AI quan trọng chỉ vì chúng diễn ra ở nhiều lượt chat khác nhau.

3. Ghi lại thông tin cho từng tương tác AI

Với mỗi tương tác có ý nghĩa, ghi các thông tin sau:

1. **Interaction ID:** mã duy nhất, tăng dần hoặc theo quy ước hiện có.
2. **AI tool:** tên công cụ, ví dụ GitHub Copilot.
3. **Date and time:** ngày giờ thực tế nếu xác định được; sử dụng múi giờ Việt Nam (UTC+7) khi phù hợp.
4. **Context/Task:** mục tiêu công việc.
5. **User prompt:** prompt thực tế đã gửi cho AI.
6. **AI output:** đầu ra thực tế của AI, có thể tóm tắt nếu đầu ra quá dài nhưng phải giữ các kết luận quan trọng và những thay đổi được đề xuất.
7. **Tệp và thao tác:** các file đã đọc, tạo, sửa hoặc xóa; lệnh đã chạy; test đã thực thi nếu có.
8. **Actual result:** kết quả công việc thực tế, phân biệt rõ giữa đề xuất, thay đổi đã thực hiện và hành động đã xác minh.
9. **Rà soát của người dùng:** những điểm tôi đã tự kiểm tra, sửa, chấp nhận hoặc chưa xác minh.
10. **Limitations:** các giới hạn, lỗi, thông tin thiếu hoặc việc chưa hoàn thành.
11. **Related artifacts:** đường dẫn tới test plan, execution results, evidence, bug report hoặc tài liệu liên quan nếu có.

Quy tắc về prompt và đầu ra

- Nếu lịch sử chat hiện tại còn chứa prompt và đầu ra gốc, hãy ghi lại chính xác prompt đó; với đầu ra dài, ghi tóm tắt trung thực và dẫn tới file kết quả liên quan.
- Nếu lịch sử cũ không còn truy cập được, không tự dựng lại prompt hoặc câu trả lời như thể đó là nguyên văn. Ghi rõ `Prompt gốc không còn truy cập được` hoặc `Chỉ ghi nhận được bản tóm tắt`.
- Nếu không xác định được ngày giờ chính xác, ghi `Không xác định được`; không tự tạo timestamp.
- Nếu một hành động chỉ được đề xuất nhưng chưa được thực hiện, phải ghi là `Đề xuất`, không phải `Đã thực hiện`.
- Nếu script đã chạy nhưng có test FAIL, giữ đúng trạng thái FAIL.
- Nếu test bị BLOCKED hoặc NOT AUTOMATED, không ghi là đã hoàn thành.
- Không khẳng định đã đọc, sửa, chạy, kiểm tra hoặc xác minh một file nếu không có căn cứ thực tế.

4. Cập nhật AI Audit theo kiểu nối tiếp

Trong `ai-audit.md`:

- Giữ nguyên các nội dung hợp lệ đã được ghi từ session trước.
- Chỉ bổ sung các tương tác mới của session hiện tại.
- Không ghi đè toàn bộ file cũ.
- Không tự xóa các mục cũ vì cho rằng chúng trùng lặp hoặc không quan trọng.
- Kiểm tra Interaction ID để tránh tạo hai mục giống nhau.
- Nếu phát hiện một mục cũ sai hoặc cần sửa, ghi rõ nội dung nào được sửa và lý do; không âm thầm thay đổi lịch sử.
- Giữ cấu trúc dễ đọc bằng Markdown và sử dụng cùng quy ước trình bày trong toàn bộ file. Có thể dùng bảng tổng quan với các cột:

```
Interaction IDDate/timeAI toolTaskResultHuman reviewArtifacts
```

Sau bảng tổng quan, bổ sung phần chi tiết cho từng tương tác, gồm prompt và đầu ra AI theo khả năng truy cập thực tế.

> Bảo đảm AI Audit đáp ứng yêu cầu HW02: khai báo công cụ AI, thời gian, prompt và đầu ra cho các tương tác có thể xác minh. Không thay thế toàn bộ lịch sử bằng một đoạn tổng kết chung chung.
>
> ## 5. Phân biệt các công cụ AI
>
> Tôi có sử dụng ChatGPT và GitHub Copilot trong quá trình làm HW02.
>
> - Chỉ ghi GitHub Copilot cho tương tác thực hiện bằng Copilot.
> - Chỉ ghi ChatGPT cho tương tác thực hiện bằng ChatGPT.
> - Nếu không có lịch sử của một công cụ trong session hiện tại, không tự tạo tương tác cho công cụ đó.
> - Nếu có bằng chứng về tương tác ở các session trước trong file audit, hãy giữ nguyên và liên kết đúng với những tương tác đó.
>
> ## 6. Kiểm tra sau khi cập nhật
>
> Sau khi ghi audit:
>
> 1. Kiểm tra Markdown và các bảng không bị lỗi định dạng.
> 2. Kiểm tra đường dẫn đến các file liên quan nếu có thể.
> 3. Đảm bảo mục vừa thêm không trùng với mục trước.
> 4. Liệt kê các tương tác đã ghi nhận.
> 5. Nêu rõ những prompt, đầu ra, ngày giờ hoặc thông tin nào chưa thể xác minh để tôi bổ sung thủ công.
> 6. Xác nhận đường dẫn file `ai-audit.md` đã cập nhật. Không sửa source code, database, test plan, execution results hoặc evidence chỉ để phục vụ việc audit. Không reset database, không commit hoặc push Git.
>
> **Bắt đầu bằng việc đọc AI Audit hiện tại và rà soát lịch sử session đang truy cập được. Sau đó, trực tiếp cập nhật `ai-audit.md` và báo cáo kết quả. Không chỉ đưa ra mẫu hoặc hướng dẫn.**

- **Tóm tắt đầu ra AI:** Tìm thấy repository HW02 ở `D:\HW02`, đọc audit hiện có và xác nhận các interaction IDs tới COPILOT-10. Kiểm tra trạng thái Git, evidence index, FR reports và sự hiện diện của artifacts trong EShop/HW02. Nối thêm COPILOT-11–14 và ghi rõ artifacts còn hiện diện cùng giới hạn về automation scripts.
- **Tệp và thao tác:** Đọc `D:\HW02\ai-audit.md` và git diff/status trước khi sửa; đọc README và cấu trúc file liên quan; chỉ nối phần nhật ký mới vào `D:\HW02\ai-audit.md`.
- **Kết quả thực tế:** Cập nhật audit theo kiểu nối tiếp. Không sửa test plan, execution results, evidence, source code hay database; không commit/push. Tình trạng `ai-audit.md` đã có thay đổi trước lượt này và các thay đổi đó được giữ lại. Xác minh reports/index cùng 46 ảnh hiện có trong HW02; không tìm thấy thư mục automation scripts trong hai workspace.
- **Rà soát của người dùng:** Các ID COPILOT-11–14 tiếp nối ID hiện có; timestamp được lấy từ user messages, không suy đoán. Phần đính kèm COPILOT-12 được mô tả đúng là attachment và tóm tắt, không trình bày như prompt đã chép nguyên văn.
- **Giới hạn/chưa xác minh:** Không có transcript đầy đủ của mọi đầu ra AI trước các lượt có timestamp. COPILOT-12 ghi nhận attachment theo tên và tóm tắt nội dung; mã automation không còn trong workspace để kiểm tra. Không xác định được thời điểm/cách evidence được lưu vào HW02.
- **Tài liệu liên quan:** `D:\HW02\ai-audit.md`; `D:\HW02\evidence\evidence-index.md`; ba FR execution reports và thư mục evidence per-case.

---

5. Tổng kết và giới hạn của AI Audit

5.1. Phạm vi các tương tác đã ghi nhận

- **ChatGPT:** Hỗ trợ phân tích yêu cầu bài tập, tổ chức repository HW02, thiết kế Domain Testing và BVA ban đầu cho FR-01 và FR-09, cùng hướng dẫn ghi nhận kết quả và báo cáo.
- **GitHub Copilot:** Hỗ trợ tạo và kiểm tra Agent Skill; phân tích/đối chiếu FR-01; tổ chức evidence; cập nhật các tài liệu demo; lập Bug Reports và AI Gap Analysis; nối tiếp nhật ký AI Audit.
- **Các feature được đề cập:** FR-01, FR-09, FR-17 và FR-11. Mức độ chi tiết và khả năng truy vết prompt/đầu ra khác nhau giữa các feature và session.

5.2. Nguyên tắc diễn giải kết quả

Các đầu ra thiết kế không phải là kết quả thực thi. Trạng thái PASS/FAIL/BLOCKED/NOT AUTOMATED phải lấy từ báo cáo thực thi và evidence tương ứng, không suy ra từ việc AI đã tạo test case hoặc file. Các lỗi ứng dụng, giới hạn test design, sự cố môi trường và test bị chặn cần được phân biệt rõ.

5.3. Giới hạn còn lại

Lưu ý về các session Copilot cũ và quá trình viết/chạy auto test

Trong quá trình lập AI Audit, lịch sử của session GitHub Copilot dùng để xây dựng và chạy bộ kiểm thử tự động ban đầu không còn truy cập đầy đủ trong ngữ cảnh audit hiện tại. Vì vậy, không thể khôi phục hoặc xác minh toàn bộ prompt, phản hồi của AI, các bước tạo/sửa script, quá trình debug và những quyết định được đưa ra trong session cũ. Đây là giới hạn của phần lịch sử hiện có để đối chiếu; không có nghĩa là các tương tác đó chắc chắn đã không diễn ra.

Audit hiện chỉ ghi nhận được những nội dung có căn cứ từ lịch sử còn truy cập và artifacts còn hiện diện. Cụ thể, các mục COPILOT-11–COPILOT-13 ghi lại việc giải thích cấu hình screenshot, bổ sung evidence và chạy lại 31 UI test thuộc FR-01/FR-09/FR-17 theo log của phiên. Tuy nhiên, mã automation không còn tìm thấy trong các workspace đã kiểm tra, nên không thể dùng chúng để tái dựng đầy đủ giai đoạn viết và chạy bộ test ban đầu.

Do đó, phần audit liên quan đến quá trình tạo auto test ban đầu có thể chưa đầy đủ. Báo cáo không tự bổ sung prompt, timestamp, nội dung mã hoặc kết luận về mức độ đóng góp của AI khi không có bằng chứng; các kết quả kiểm thử chỉ được mô tả theo execution report và evidence còn đối chiếu được.

- Một số tương tác ChatGPT không còn timestamp chính xác hoặc chỉ còn bản tóm tắt thay vì transcript đầy đủ.
- Một số prompt Copilot được lưu dưới dạng tóm tắt nội dung attachment thay vì bản văn nguyên văn; điều này được ghi chú tại từng mục.
- Một số timestamp được xác định từ thời điểm hiển thị trong user message; các thời điểm không có căn cứ được để là chưa xác định.
- Các kết quả/đường dẫn được xác nhận tại thời điểm ghi nhật ký không đồng nghĩa với việc tất cả artifacts đã được kiểm tra lại ở cùng một thời điểm cuối cùng.
- Video demo Skill và Git commit log cần được cập nhật khi hoàn thành/theo phương án nhóm thống nhất; không khẳng định hai phần này đã hoàn tất nếu chưa có bằng chứng.

5.4. Trách nhiệm rà soát của người học

Trước khi nộp, cần đối chiếu các đường dẫn và thông tin còn thiếu; xác minh prompt, timestamp hoặc đầu ra nếu có thể truy xuất lại lịch sử; kiểm tra sự nhất quán giữa AI Audit, AI Gap Analysis, Bug Reports, execution results và evidence. Báo cáo này ghi lại những gì có thể truy vết từ tài liệu/lịch sử còn truy cập được, không thay thế việc rà soát cuối cùng của người học.